USBD40: Register a freight in the database - Sprint 3

Este documento descreve a implementação, configuração de dados e testes realizados para a **USBD40**.

1. Descrição da User Story

- As a Freight Manager, I want to register a freight in the database. This includes the wagons involved. This is only possible if there are enough free wagons of the given type at the freight's departure facility.

Critérios de Aceitação e Regras de Negócio:

- Um freight é identificado por um ID único.
- O sistema deve validar se existem vagões suficientes do tipo solicitado na instalação de origem.
- O sistema deve impedir a duplicação de IDs de freight.
- O sistema deve validar que a quantidade solicitada é maior que zero.
- Todos os vagões associados ao freight devem ser registados na tabela de associação `Freight_Wagon`.
- A implementação deve ser feita através de uma procedure PL/SQL.

---

2. Explicação do Script (Procedure PL/SQL)

A lógica central da **USBD40** foi encapsulada na procedure `pr_register_freight`. Esta abordagem permite centralizar as regras de negócio e garantir a integridade dos dados ao criar um novo transporte de carga.

Detalhes da Implementação:

1. **Assinatura da Procedure:** A procedure aceita 6 parâmetros: ID do freight, instalação de origem, instalação de destino, data, modelo de vagão e quantidade solicitada.
2. **Validação de Parâmetros Obrigatórios:** Verifica se algum parâmetro essencial é `NULL` ou se a quantidade é menor ou igual a zero. Lança exceção `-20010` em caso afirmativo.
3. **Verificação de ID Duplicado:** Consulta a tabela `Freight` para verificar se o ID já existe. Lança exceção `-20011` se duplicado.
4. **Verificação de Disponibilidade de Vagões:**
    - Conta quantos vagões do modelo especificado estão disponíveis na instalação de origem.
    - Compara com a quantidade solicitada.
    - Lança exceção `-20012` se não houver vagões suficientes, incluindo os valores disponíveis e solicitados na mensagem de erro.
5. **Inserção do Freight:** Cria o registo na tabela `Freight` com a data, origem e destino.
6. **Associação de Vagões:**
    - Usa um cursor `FOR` loop ordenado por `wagon_number` para garantir determinismo.
    - Utiliza uma variável local `v_qty_to_assign` (inicializada com o valor de `p_qty`) para controlar o loop, evitando modificar o parâmetro `IN` diretamente.
    - Insere cada vagão selecionado na tabela `Freight_Wagon`.
    - Decrementa a variável local até atingir a quantidade desejada.
7. **Transação:** Realiza `COMMIT` em caso de sucesso ou `ROLLBACK` em caso de erro.

Nota Técnica: A implementação usa `v_qty_to_assign` em vez de modificar `p_qty` diretamente, respeitando a restrição do PL/SQL que proíbe alterar parâmetros `IN`.

Código: `USBD40-script.sql`

```
CREATE OR REPLACE PROCEDURE pr_register_freight(
    p_freight_id IN NUMBER,
    p_origin_id  IN NUMBER,
    p_dest_id    IN NUMBER,
    p_date       IN DATE,
    p_model_id   IN NUMBER,
    p_qty        IN NUMBER
) IS
    v_available_wagons NUMBER;
    v_count            NUMBER;
    v_qty_to_assign    NUMBER; -- Variável local para controlo do loop
BEGIN
    -- 1. Validação de parâmetros
    IF p_freight_id IS NULL OR p_origin_id IS NULL OR p_dest_id IS NULL OR p_qty <= 0 THEN
        RAISE_APPLICATION_ERROR(-20010, 'USBD40: Parâmetros inválidos ou quantidade deve ser maior que zero.');
END IF;

    -- 2. Validar se Freight ID já existe
SELECT COUNT(*) INTO v_count FROM Freight WHERE id = p_freight_id;
IF v_count > 0 THEN
        RAISE_APPLICATION_ERROR(-20011, 'USBD40: Freight ID ' || p_freight_id || ' já existe.');
END IF;
```

```
    -- 3. Verificar disponibilidade de vagões na origem
SELECT COUNT(*)
INTO v_available_wagons
FROM Wagon
WHERE current_facility_id = p_origin_id
  AND model_id = p_model_id;

IF v_available_wagons < p_qty THEN
        RAISE_APPLICATION_ERROR(-20012, 'USBD40: Quantidade insuficiente. Disponíveis: ' || v_available_wagons || ', Solicitados: '
END IF;

    -- 4. Inserir Freight
INSERT INTO Freight (id, freight_date, origin_facility_id, dest_facility_id)
VALUES (p_freight_id, p_date, p_origin_id, p_dest_id);

-- 5. Associar Vagões
-- Inicializamos a variável local com o valor do parâmetro de entrada
v_qty_to_assign := p_qty;

FOR r IN (
        SELECT wagon_number
        FROM Wagon
        WHERE current_facility_id = p_origin_id
          AND model_id = p_model_id
        ORDER BY wagon_number
    ) LOOP
        -- Sai do loop quando já tivermos atribuído a quantidade necessária
        EXIT WHEN v_qty_to_assign <= 0;

        -- Insere na tabela de associação
INSERT INTO Freight_Wagon (freight_id, wagon_number)
VALUES (p_freight_id, r.wagon_number);

-- Decrementa a variável local (permitido) em vez do parâmetro (proibido)
v_qty_to_assign := v_qty_to_assign - 1;
END LOOP;

    DBMS_OUTPUT.PUT_LINE('Sucesso: Freight ' || p_freight_id || ' criado com ' || p_qty || ' vagões.');
COMMIT;

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END;
/
```

3. Explicação de Cada Teste

Para validar a User Story USBD40, foram desenvolvidos 5 cenários de teste que cobrem inserções válidas, violações de integridade e regras de negócio.

Teste 1: Inserção Bem-Sucedida

Objetivo: Verificar se é possível criar um novo freight válido quando existem vagões suficientes na instalação de origem.

Descrição: O teste garante primeiro que não existem conflitos prévios e depois tenta criar o freight 9001, solicitando 1 vagão do modelo 1104 da instalação de Leixões (ID 50) para Valença (ID 11).

Resultado Esperado: A procedure deve executar sem erros. Deve ser criado 1 registo na tabela `Freight` e 1 registo na tabela `Freight_Wagon`.

```
 SET SERVEROUTPUT ON;
DECLARE
v_freight_id NUMBER := 9001; -- ID Novo manual
    v_origin     NUMBER := 50;   -- Leixões (Tem stock)
    v_dest       NUMBER := 11;   -- Valença
    v_model      NUMBER := 1104; -- Modelo existente
    v_qty        NUMBER := 1;    -- Pedir 1 vagão

    v_check_f    NUMBER;
    v_check_w    NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- USBD40: TESTE 1 (Sucesso) ---');

    -- Limpeza preventiva
DELETE FROM Freight_Wagon WHERE freight_id = v_freight_id;
DELETE FROM Freight WHERE id = v_freight_id;

-- Executar Procedure
pr_register_freight(
        p_freight_id => v_freight_id,
        p_origin_id  => v_origin,
        p_dest_id    => v_dest,
        p_date       => SYSDATE + 2,
        p_model_id   => v_model,
        p_qty        => v_qty
    );

    -- Validação
SELECT COUNT(*) INTO v_check_f FROM Freight WHERE id = v_freight_id;
SELECT COUNT(*) INTO v_check_w FROM Freight_Wagon WHERE freight_id = v_freight_id;

IF v_check_f = 1 AND v_check_w = v_qty THEN
        DBMS_OUTPUT.PUT_LINE('RESULTADO: SUCESSO. Freight criado e vagões associados.');
ELSE
        DBMS_OUTPUT.PUT_LINE('RESULTADO: FALHA. Contagem incorreta.');
END IF;

ROLLBACK;
END;
/
```

Teste 2: Falha - Sem Stock na Estação

Objetivo: Verificar se o sistema impede a criação de um freight quando não existem vagões disponíveis na instalação de origem.

Descrição: O teste tenta criar um freight com origem na estação de Valença (ID 11), que não possui vagões do modelo 1104 em stock.

Resultado Esperado: A procedure deve lançar a exceção `-20012` (Quantidade Insuficiente), indicando que não há vagões disponíveis na instalação.

```
 SET SERVEROUTPUT ON;
DECLARE
v_freight_id NUMBER := 9002;
    v_origin     NUMBER := 11;   -- Valença (Sem stock)
    v_model      NUMBER := 1104;
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- USBD40: TESTE 2 (Falha - Sem Stock na Estação) ---');

    pr_register_freight(
        p_freight_id => v_freight_id,
        p_origin_id  => v_origin,
        p_dest_id    => 50,
        p_date       => SYSDATE + 2,
        p_model_id   => v_model,
        p_qty        => 1
    );

    DBMS_OUTPUT.PUT_LINE('RESULTADO: FALHA. O sistema permitiu criar sem vagões.');
EXCEPTION
    WHEN OTHERS THEN
        IF SQLCODE = -20012 THEN
            DBMS_OUTPUT.PUT_LINE('RESULTADO: SUCESSO. Erro de falta de vagões detetado.');
ELSE
            DBMS_OUTPUT.PUT_LINE('RESULTADO: FALHA. Erro inesperado: ' || SQLERRM);
END IF;
END;
/
```

Teste 3: Falha - Quantidade Exagerada

Objetivo: Verificar se o sistema impede a criação de um freight quando a quantidade solicitada excede o stock disponível.

Descrição: O teste tenta criar um freight solicitando 1000 vagões, quantidade muito superior ao stock real disponível em Leixões.

Resultado Esperado: A procedure deve lançar a exceção `-20012` (Quantidade Insuficiente), mostrando na mensagem de erro a quantidade disponível vs. solicitada.

```
SET SERVEROUTPUT ON;
DECLARE
v_freight_id NUMBER := 9003;
    v_origin     NUMBER := 50;
    v_model      NUMBER := 1104;
    v_qty        NUMBER := 1000; -- Exagerado
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- USBD40: TESTE 3 (Falha - Quantidade Exagerada) ---');

    pr_register_freight(
        p_freight_id => v_freight_id,
        p_origin_id  => v_origin,
        p_dest_id    => 11,
        p_date       => SYSDATE + 2,
        p_model_id   => v_model,
        p_qty        => v_qty
    );

    DBMS_OUTPUT.PUT_LINE('RESULTADO: FALHA. O sistema permitiu pedir stock inexistente.');
EXCEPTION
    WHEN OTHERS THEN
        IF SQLCODE = -20012 THEN
            DBMS_OUTPUT.PUT_LINE('RESULTADO: SUCESSO. Erro de quantidade insuficiente detetado.');
ELSE
            DBMS_OUTPUT.PUT_LINE('RESULTADO: FALHA. Erro inesperado: ' || SQLERRM);
END IF;
END;
/
```

Teste 4: Falha - ID Duplicado

Objetivo: Verificar se o sistema impede a criação de um freight com um ID que já existe na base de dados.

Descrição: O teste tenta criar um freight usando o ID 2001, que já existe no sistema (inserido no USBD32).

Resultado Esperado: A procedure deve lançar a exceção `-20011` (ID Duplicado), confirmando que a validação funciona corretamente.

```
SET SERVEROUTPUT ON;
DECLARE
    v_existing_id NUMBER := 2001;
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- USBD40: TESTE 4 (Falha - ID Duplicado) ---');

    pr_register_freight(
        p_freight_id => v_existing_id,
        p_origin_id  => 50,
        p_dest_id    => 11,
        p_date       => SYSDATE + 2,
        p_model_id   => 1104,
        p_qty        => 1
    );

    DBMS_OUTPUT.PUT_LINE('RESULTADO: FALHA. O sistema permitiu ID duplicado.');
EXCEPTION
    WHEN OTHERS THEN
        IF SQLCODE = -20011 THEN
            DBMS_OUTPUT.PUT_LINE('RESULTADO: SUCESSO. Erro de ID duplicado detetado.');
        ELSE
            DBMS_OUTPUT.PUT_LINE('RESULTADO: FALHA. Erro inesperado: ' || SQLERRM);
        END IF;
END;
/
```

Teste 5: Falha - Quantidade Negativa

Objetivo: Verificar se o sistema valida corretamente os parâmetros de entrada, impedindo valores inválidos.

Descrição: O teste tenta criar um freight com uma quantidade negativa (-5), que viola a regra de negócio.

Resultado Esperado: A procedure deve lançar a exceção `-20010` (Parâmetros Inválidos), confirmando que a validação inicial funciona corretamente.

```
 SET SERVEROUTPUT ON;
DECLARE
v_freight_id NUMBER := 9999;
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- USBD40: TESTE 5 (Falha - Quantidade Negativa) ---');

    pr_register_freight(
        p_freight_id => v_freight_id,
        p_origin_id  => 50,
        p_dest_id    => 11,
        p_date       => SYSDATE + 2,
        p_model_id   => 1104,
        p_qty        => -5 -- Inválido
    );

    DBMS_OUTPUT.PUT_LINE('RESULTADO: FALHA. O sistema aceitou quantidade negativa.');
EXCEPTION
    WHEN OTHERS THEN
        IF SQLCODE = -20010 THEN
            DBMS_OUTPUT.PUT_LINE('RESULTADO: SUCESSO. Erro de parâmetros detetado.');
ELSE
            DBMS_OUTPUT.PUT_LINE('RESULTADO: FALHA. Erro inesperado: ' || SQLERRM);
END IF;
END;
/
```